

Project AtlasCare

Agentic AI for Customer Contact

1. Client Context

Acme Retail Co. is a mid-sized online retailer with approximately 2 million monthly active users and a product catalogue of roughly 150,000 SKUs spanning apparel, electronics, and home goods.

Their contact centre receives around 18,000 customer interactions per day across chat, email, and voice. Tier-1 support today is a hybrid of rule-based chatbots and human agents, and it is under pressure on three fronts:

Pain Point	Detail
Rising cost per contact	Rule-based bots require constant manual maintenance. Human fallback rate is too high for simple queries that should be self-serve.
Inconsistent CX	Different Bot versions and agent styles produce an inconsistent customer experience. Brand and compliance teams have flagged this.
Slow multi-step resolution	Compound requests — e.g. cancel one item, refund to a different card, reship the rest to a new address — require multiple handoffs and manual co-ordination, leading to long handling times and errors.

2. Your Assignment

You are being asked to design and build Project AtlasCare — an Agentic AI layer that can autonomously handle a meaningful percentage of Tier-1 interactions, while safely escalating those it cannot.

This is not a proof-of-concept exercise. We are not evaluating whether your agent gives the right answer on the happy path. We are evaluating whether you understand what it takes to take a working prototype and make it production-grade — observable, safe, scalable, and auditable — in a live enterprise environment.

Your system must integrate with Acme's existing enterprise stack and be operable by their SRE team and auditable by their Risk & Compliance team. You will be provided with a shared LLM endpoint and mock data for all downstream systems.

2.1 Enterprise Stack

Your agent must interact with the following systems using the mock data provided in SharePoint link.

The mock data files contain schema and sample data, feel free to generate more synthetic data as per schema for your own testing. The generated synthetic data must adhere to the schema provided. For this assignment, you can either create API or directly use Json files provided for tool calling.

System	What It Provides
OMS (Order Management)	Order status, line-item details, partial cancellation, address updates.
CRM (Customer Records)	Customer profile, interaction history, case creation and handoff.
KB (Knowledge Base)	Policy documents, FAQ, escalation thresholds, return rules.

System	What It Provides
Payments Gateway	Refund initiation. Auto-refund limit: ₹25,000. Amounts above this threshold must never be processed autonomously.

3. The Three Test Journeys

Your system will be evaluated against the three journeys below. All three must work end-to-end. Each has specific hard constraints — read them carefully.

Candidates can add any other journeys as well apart from what is mentioned in below table. It will not have any negative impact in scoring.

	Journey	User Says	What the System Must Do
J1	Simple Tracking	"Where is my order ORD-XXXXX?"	Respond with status and tracking detail in under 3 seconds. Use exactly one OMS call. Do not escalate. Do not hallucinate any order details.
J2	Multi-Step Compound Request	"Cancel item 2 from order ORD-XXXXX, refund it to my HDFC card, and ship the other two items to my office address instead."	Plan a multi-step sequence. Execute: (1) partial cancellation on OMS, (2) refund initiation on Payments, (3) address update on OMS. Record every tool outcome — success or failure — in the trace. Confirm accurately to the user.
J3	Escalation with Audit Trail	"I want a full refund of ₹42,000 for order ORD-XXXXX, because the laptop arrived damaged."	Detect that ₹42,000 exceeds the ₹25,000 auto-refund threshold. Do NOT call the Payments tool. Create a CRM case with a structured handoff summary linked to the trace_id. Return a polite holding message to the user.

4. What We Are Evaluating

We are not marking you on LLM accuracy or prompt quality alone. We are assessing whether you think like an engineer who has to deploy, operate, and be on call for an agentic system at scale — not someone who built a demo that worked once.

Your submission will be assessed across dimensions which are required for a production grade agentic AI application such as performance across business KPIs, scalable architecture, governance, security, compliance and evaluations.

5. Deliverables

5.1 Submit a single ZIP archive

The single Zip file needs to include below items mentioned in the table (**Refer section 7 for mandatory Repo structure**). The details mentioned in below table are minimum requirement for assignment, however candidates are free to add any other files as per their own solution approach and thought process.

Deliverable	Expected Content
Working code	All three journeys must run end-to-end against the mock data and LLM endpoint as mentioned in section 6 & 7. Package your entire submission

Deliverable	Expected Content
	as a single ZIP file and upload it to your SharePoint. Name the file as <code>firstname_lastname_empid.zip</code>
Architecture document (.md file format)	A concise written or diagrammatic explanation of your agent design: planning approach, tool-call strategy, guardrail implementation, and observability hooks. Max 2 pages.
KPI framework (.md file format)	A list of the KPIs you would track in production, grouped by layer (business, quality, safety, operational).
Test plan summary (.txt or .docx format)	A description of your testing approach: what is covered, what is deliberately out of scope for this submission, and what you would add before going live.
README	Setup, environment variables, how to run the three journeys, and any known limitations.

Submit a Video debrief

A video debriefs (<20 mins) of the solution built highlighting the approach, problem understanding, architecture & demo for the final submission. In case no submission is made, the candidature will not be taken ahead for the MLE Track Transfer process further.

6. Resources Provided

The following will be shared with you separately via email:

- OMS data schema — order structure with full line-item field definitions
- CRM data schema — customer profile and interaction history structure
- KB schema — policy document structure, refund threshold rules, escalation SLA fields
- Payment's config schema — refund request and response structure

7. Prerequisites & Requirements

7.0 Mandatory Rules

The below rules are non-negotiable. Your submission will not be assessed if either is violated.

Rule 1 — Repository Structure

Your ZIP must follow this structure exactly:

```
CAND-XXXX.zip
├─ main.py           ← at ZIP root
├─ requirements.txt  ← at ZIP root
└─ (everything else) ← your choice
```

Rule 2 — Interface Contract

Your agent must expose these two endpoints with the exact structure shown:

POST /query

```
Request:
{"message": "string", "session_id": "string"}
```

```
Response (HTTP 200):
{
  "response": "string",
  "trace": {
    "trace_id": "string",
    "session_id": "string",
    "latency_ms": integer,
    "tool_calls": []
  }
}
```

GET /health → HTTP 200

7.1 Technical Environment

The submission should adhere to below requirements

Requirement	Specification	Notes
Python	3.10, 3.11, or 3.12	No other versions supported
Minimum packages	fastapi, uvicorn, httpx	Must be in requirements.txt
LLM provider	Gemini 2.5 Flash (free tier)	Sign up at aistudio.google.com Use the OpenAI-compatible endpoint Read all config from environment variables
OS	Windows, Mac, or Linux	All supported

You need to utilise Gemini 2.5 Flash LLM end point. Do not use any other LLM available. Free tier at aistudio.google.com provides 1,500 requests/day — sufficient for development and testing.

Refer to the doc for more information: [How to Get Free AI API Access with Google Gemini and Where It Actually Makes Sense to Use It](#) | [Kathan N. Patel](#)

Good luck!

For questions reach out to ADS Track transfer Email